



Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Grupo 5k2

Proyecto Segundo Parcial: Bases de Datos Distribuidas

Bases de Datos Avanzadas

Docente: José Saúl De Lira Miramontes

Osvaldo Hernández Juarez
Matrícula: 385493

Diego Flores Verdad Grijalva
Matrícula: 385660

Luis Felipe Domínguez Chávez
Matrícula: 385500

4 de noviembre de 2026

Índice

1 Introducción **2**

2 Objetivo **2**

3 Entorno de trabajo **2**

4 Configurar un Cluster en Mongo sobre Docker con los siguientes nodos: configsvr, shard1, shard2, shard3, Replica set, mongos router **2**

4.1 Arquitectura 2
4.2 Docker Compose 3
4.3 Levantamiento de los contenedores 6
4.4 Inicialización de los replica sets 6
4.5 Registro de los shards 9

5 Utilizar la base de datos sample_mflix y la colección movies **9**

5.1 Importación de los datos 9
5.2 Monitoreo de la distribución 10

6 Agregar a los documentos de películas un id_movie **11**

7 Definir el Shard Key de la colección Tipo: “Hashed” **13**

8 Replica sets de los shards **15**

8.1 Estado de los replica sets 15
8.2 Los SECONDARY tienen los mismos datos 16
8.3 Prueba de failover 17

9 Mostrar un ejemplo de balanceo de carga entre shards **19**

10 Realizar operaciones CRUD sobre la base de datos distribuida **21**

10.1 Create 21
10.2 Read 21
10.3 Update 21
10.4 Delete 21

11 Conclusión **22**

Referencias **22**

1 Introducción

En la actividad anterior se configuró un cluster de MongoDB con sharding sobre Docker y una colección pequeña de usuarios. En este proyecto el mismo esquema se aplicó a una base de datos real, sample_mflix, cuya colección movies contiene 21 349 películas. Cada shard se configuró como un replica set de tres miembros, se agregó a cada película un identificador numérico, id_movie, que se usó como shard key de tipo hashed, y sobre la colección distribuida se realizaron operaciones CRUD, monitoreando después de cada una cómo quedan repartidos los documentos entre los shards. También se muestra cómo el balanceador reparte la carga cuando se agrega un shard nuevo a un cluster que ya tiene datos.

Los archivos del proyecto (Docker Compose, scripts de configuración, monitoreo, balanceo y operaciones CRUD) se encuentran en el repositorio https://github.com/LF-Dguez/bases_de_datos_distribuidas_MongoDB.

2 Objetivo

Implementar soluciones para Bases de Datos Distribuidas.

3 Entorno de trabajo

El proyecto se realizó en Windows 11 con Docker Desktop y Docker Compose v2. Los once nodos del cluster usan la imagen oficial `mongo:8.0`. Los scripts se escribieron en JavaScript y se ejecutaron con el `mongosh` que incluye la imagen, dentro del contenedor del router; la importación de los datos se hizo con `mongoimport`, que también viene en la imagen.

4 Configurar un Cluster en Mongo sobre Docker con los siguientes nodos: configsvr, shard1, shard2, shard3, Replica set, mongos router

4.1 Arquitectura

El cluster está formado por once contenedores:

- `configsvr`: config server. Guarda los metadatos del cluster, como la lista de shards y los rangos de datos (chunks) de cada uno. Es un replica set de un miembro.
- `shard1`, `shard2` y `shard3`: cada shard es un replica set de tres miembros (por ejemplo, `shard1-1`, `shard1-2` y `shard1-3`). Uno de ellos es el PRIMARY, que recibe las escrituras, y los otros dos son SECONDARY y mantienen una copia

de los mismos datos. Así cada parte de la colección está guardada tres veces y el shard sigue funcionando si falla uno de sus miembros.

- `mongos`: router. Las aplicaciones se conectan a él y él envía cada operación al shard que corresponde. No guarda datos.

4.2 Docker Compose

```
# Cluster de MongoDB con sharding para sample_mflix.movies
#
#   configsvr          config server (replica set configReplSet, 1 miembro)
#   shard1-1..shard1-3 shard 1 (replica set shard1ReplSet, 3 miembros)
#   shard2-1..shard2-3 shard 2 (replica set shard2ReplSet, 3 miembros)
#   shard3-1..shard3-3 shard 3 (replica set shard3ReplSet, 3 miembros)
#   mongos             router, unico nodo publicado (localhost:27030)
#
# Cada mongod limita su cache a 0.25 GB para que los 10 procesos quepan en
# la memoria de Docker Desktop.

x-shard: &shard
  image: mongo:8.0
  networks:
    - mongo-cluster

services:
  configsvr:
    image: mongo:8.0
    container_name: configsvr
    command: >-
      mongod --configsvr --replSet configReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - configsvr_data:/data/configdb
    networks:
      - mongo-cluster

  shard1-1:
    <<: *shard
    container_name: shard1-1
    command: >-
      mongod --shardsvr --replSet shard1ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard1-1_data:/data/db
  shard1-2:
    <<: *shard
    container_name: shard1-2
    command: >-
      mongod --shardsvr --replSet shard1ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard1-2_data:/data/db
  shard1-3:
    <<: *shard
    container_name: shard1-3
```

```
    command: >-
      mongod --shardsvr --replSet shard1ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard1-3_data:/data/db

  shard2-1:
    <<: *shard
    container_name: shard2-1
    command: >-
      mongod --shardsvr --replSet shard2ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard2-1_data:/data/db
  shard2-2:
    <<: *shard
    container_name: shard2-2
    command: >-
      mongod --shardsvr --replSet shard2ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard2-2_data:/data/db
  shard2-3:
    <<: *shard
    container_name: shard2-3
    command: >-
      mongod --shardsvr --replSet shard2ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard2-3_data:/data/db

  shard3-1:
    <<: *shard
    container_name: shard3-1
    command: >-
      mongod --shardsvr --replSet shard3ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard3-1_data:/data/db
  shard3-2:
    <<: *shard
    container_name: shard3-2
    command: >-
      mongod --shardsvr --replSet shard3ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard3-2_data:/data/db
  shard3-3:
    <<: *shard
    container_name: shard3-3
    command: >-
      mongod --shardsvr --replSet shard3ReplSet --port 27017
      --bind_ip_all --wiredTigerCacheSizeGB 0.25
    volumes:
      - shard3-3_data:/data/db

  mongos:
    image: mongo:8.0
```

```
    container_name: mongos
    command: >-
      mongos --configdb configReplSet/configsvr:27017 --port 27017
      --bind_ip_all
    ports:
      - "27030:27017"
    volumes:
      - ./configuracion:/scripts/configuracion:ro
      - ./monitoreo:/scripts/monitoreo:ro
      - ./replicacion:/scripts/replicacion:ro
      - ./balanceo:/scripts/balanceo:ro
      - ./crud:/scripts/crud:ro
      - ./datos:/datos:ro
    depends_on:
      - configsvr
    networks:
      - mongo-cluster

volumes:
  configsvr_data:
  shard1-1_data:
  shard1-2_data:
  shard1-3_data:
  shard2-1_data:
  shard2-2_data:
  shard2-3_data:
  shard3-1_data:
  shard3-2_data:
  shard3-3_data:

networks:
  mongo-cluster:
```

Los puntos principales de la configuración son:

- El config server se inicia con `--configsvr` y los miembros de los shards con `--shardsvr`. Con `--replSet` cada uno recibe el nombre de su replica set; los tres miembros de un shard comparten el mismo nombre.
- Los nueve miembros de los shards comparten la misma configuración base, definida una sola vez con la extensión `x-shard` y reutilizada con `<<: *shard`.
- Todos escuchan en el puerto 27017 dentro de su contenedor y aceptan conexiones de los demás con `--bind_ip_all`. Cada `mongod` limita su caché a 0.25 GB con `--wiredTigerCacheSizeGB`, para que los diez procesos quepan en la memoria de Docker Desktop.
- Solo `mongos` se publica hacia el equipo, en el puerto 27030, porque el 27017 lo ocupa el MongoDB instalado en Windows.
- En `mongos` se montan las carpetas del repositorio con los scripts (`/scripts`) y con los datos (`/datos`), para ejecutarlos con las herramientas de ese contenedor.

- Cada nodo de datos tiene su propio volumen, de modo que la información sobrevive si se recrea el contenedor.

4.3 Levantamiento de los contenedores

```
docker compose up -d
docker compose ps
```

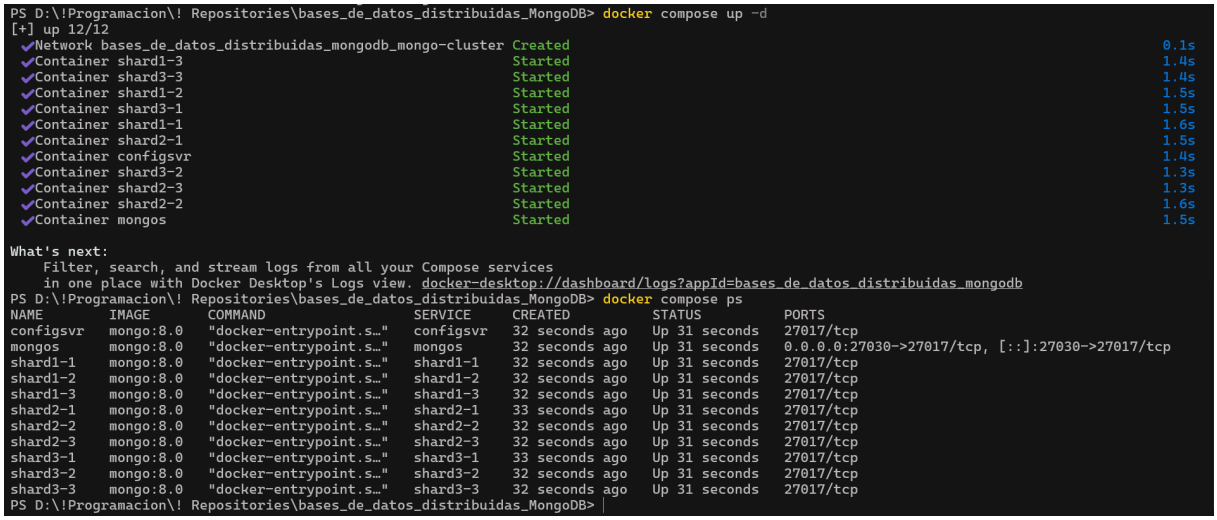


Figura 1: Creación y estado de los once contenedores.

Compose creó la red, los diez volúmenes y los once contenedores. En este punto los nodos están encendidos, pero todavía no forman un cluster.

4.4 Inicialización de los replica sets

El replica set del config server se inicializa con el comando `replSetInitiate`. El script se ejecuta desde el contenedor de `mongos` con `--nodb` y abre la conexión al config server por su nombre en la red de Docker.

```
// Paso 1. Inicializa el replica set del config server.
//
// Se ejecuta desde el contenedor de mongos, donde estan montados los
   scripts:
// docker exec mongos mongosh --nodb --quiet
//   --file /scripts/configuracion/01-init-configsvr.js
//
// --nodb porque todavia no hay nada a que conectarse por defecto; la
   conexion
// al config server se abre aqui mismo por su nombre en la red de Docker.

const uri = "mongodb://configsvr:27017/?directConnection=true";
const admin = new Mongo(uri).getDB("admin");

const res = admin.runCommand({
```

```
  replSetInitiate: {
    _id: "configReplSet",
    configsvr: true,
    members: [{ _id: 0, host: "configsvr:27017" }]
  }
});

if (res.ok === 1) {
  print("configReplSet: replica set iniciado");
} else if (res.codeName === "AlreadyInitialized") {
  print("configReplSet: ya estaba iniciado");
} else {
  printjson(res);
  quit(1);
}

// La eleccion del PRIMARY tarda unos segundos.
for (let i = 0; i < 30; i++) {
  if (admin.runCommand({ hello: 1 }).isWritablePrimary) break;
  sleep(1000);
}

const estado = admin.runCommand({ replSetGetStatus: 1 });
estado.members.forEach(m => print(`  ${m.name}  ${m.stateStr}`));
```

```
docker exec mongos mongosh --nodb --quiet --file /scripts/configuracion/01-
    init-configsvr.js
```

```
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB> docker exec mongos mongosh --nodb --quiet --file /scripts/configuracion/
01-init-configsvr.js
configReplSet: replica set iniciado
  configsvr:27017  PRIMARY
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB>
```

Figura 2: Inicialización del replica set del config server.

Para cada shard, `replSetInitiate` se ejecuta en su primer miembro con la lista de los tres. El primer miembro lleva prioridad 2 para que sea elegido PRIMARY; los otros dos quedan como SECONDARY y copian sus datos.

```
// Paso 2. Inicializa los tres shards, cada uno como replica set de 3
    miembros.
//
// docker exec mongos mongosh --nodb --quiet
//   --file /scripts/configuracion/02-init-shards.js
//
// replSetInitiate se ejecuta en el primer miembro de cada shard con la
    lista
// completa de miembros. El primero lleva priority 2 para que sea el
    PRIMARY;
// los otros dos quedan como SECONDARY y copian sus datos.

const shards = ["shard1", "shard2", "shard3"];

shards.forEach(nombre => {
  const rs = `${nombre}ReplSet`;
  const hosts = [1, 2, 3].map(n => `${nombre}-${n}:27017`);
```

```
  const uri = `mongodb://${hosts[0]}/?directConnection=true`;
  const admin = new Mongo(uri).getDB("admin");

  const res = admin.runCommand({
    replSetInitiate: {
      _id: rs,
      members: hosts.map((host, i) => ({
        _id: i,
        host: host,
        priority: i === 0 ? 2 : 1
      }))
    }
  });

  if (res.ok === 1) {
    print(`${rs}: replica set iniciado`);
  } else if (res.codeName === "AlreadyInitialized") {
    print(`${rs}: ya estaba iniciado`);
  } else {
    printjson(res);
    quit(1);
  }

  // Espera a que haya PRIMARY y a que los secundarios terminen de
     sincronizar
  for (let i = 0; i < 60; i++) {
    const st = admin.runCommand({ replSetGetStatus: 1 });
    const listos = st.ok === 1 && st.members.every(
      m => m.stateStr === "PRIMARY" || m.stateStr === "SECONDARY");
    if (listos && st.members.some(m => m.stateStr === "PRIMARY")) break;
    sleep(1000);
  }

  const estado = admin.runCommand({ replSetGetStatus: 1 });
  estado.members.forEach(m => print(`  ${m.name}  ${m.stateStr}`));
});
```

```
docker exec mongos mongosh --nodb --quiet --file /scripts/configuracion/02-
   init-shards.js
```

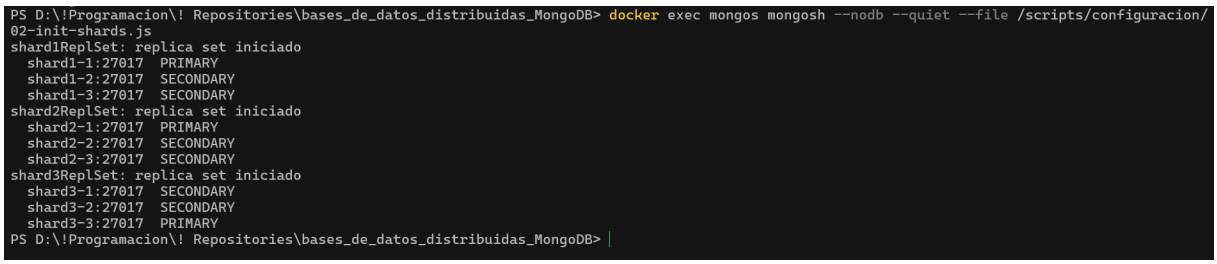


Figura 3: Inicialización de los replica sets de los tres shards.

Cada shard quedó con un miembro PRIMARY y dos SECONDARY. En shard3 la primera elección la ganó shard3-3; como shard3-1 tiene mayor prioridad, unos segundos después, ya sincronizado, toma el lugar de PRIMARY.

4.5 Registro de los shards

Se registran shard1 y shard2. El shard3 ya está inicializado, pero se agrega más adelante, con la colección ya cargada, para mostrar el balanceo de carga (sección 9).

```
// Paso 3. Registra shard1 y shard2 en el cluster.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/03-add-shards.js
//
// shard3 ya esta levantado e inicializado, pero se registra hasta la parte
    de
// balanceo (balanceo/01-agregar-shard3.js): asi se ve como el balanceador
   le
// mueve datos a un shard nuevo cuando la coleccion ya esta cargada.
//
// Cada shard se registra con el nombre de su replica set y sus tres
   miembros.

const shards = [
  "shard1ReplSet/shard1-1:27017,shard1-2:27017,shard1-3:27017",
  "shard2ReplSet/shard2-1:27017,shard2-2:27017,shard2-3:27017"
];

shards.forEach(cadena => {
  const res = sh.addShard(cadena);
  print(`addShard ${cadena.split("/")[0]}: ok ${res.ok}`);
});

print("\nShards registrados:");
db.adminCommand({ listShards: 1 }).shards
  .forEach(s => print(`  ${s._id}  ${s.host}`));
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/03-add-
   shards.js
```

```
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB> docker exec mongos mongosh --quiet --file /scripts/configuracion/03-add-
shards.js
addShard shard1ReplSet: ok 1
addShard shard2ReplSet: ok 1

Shards registrados:
  shard1ReplSet  shard1ReplSet/shard1-1:27017,shard1-2:27017,shard1-3:27017
  shard2ReplSet  shard2ReplSet/shard2-1:27017,shard2-2:27017,shard2-3:27017
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB>
```

Figura 4: Registro de shard1 y shard2.

Cada shard se registra con el nombre de su replica set y sus tres miembros, de modo que `mongos` sabe a qué nodo dirigirse si cambia el PRIMARY.

5 Utilizar la base de datos sample_mflix y la colección movies

5.1 Importación de los datos

El archivo `datos/movies.json` contiene las películas de la colección, un documento por línea en formato Extended JSON, que conserva los tipos de MongoDB como

ObjectId y fechas. Se importó a través de `mongos` con `mongoimport`:

```
docker exec mongos mongoimport --db sample_mflix --collection movies --drop
    --file /datos/movies.json
```

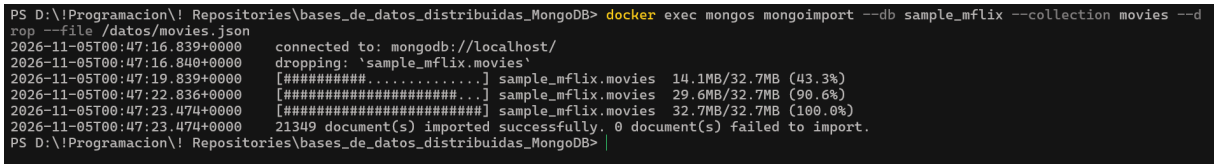


Figura 5: Importación de la colección movies.

5.2 Monitoreo de la distribución

Para revisar cómo se reparten los documentos se escribió un script de monitoreo que se ejecuta después de cada paso y de cada operación CRUD. Mientras la colección no está fragmentada, usa la etapa `$collStats` para mostrar en qué shard están los documentos. Ya fragmentada, usa `$shardedDataDistribution`, que reporta por shard los documentos que le pertenecen, su tamaño y los documentos huérfanos; además cuenta los chunks de cada shard y muestra el estado del balanceador.

```
// Monitoreo de la distribucion de sample_mflix.movies entre los shards.
//
// docker exec mongos mongosh --quiet --file /scripts/monitoreo/
   distribucion.js
//
// Se corre despues de cada paso (importacion, fragmentacion, balanceo y
   cada
// operacion CRUD) para ver como quedan repartidos los documentos.
//
// Documentos = los que le pertenecen al shard segun sus chunks.
// Huerfanos  = copias que quedaron en un shard despues de migrar un chunk
   a
//              otro. mongos nunca los devuelve en las consultas y el shard
//              los borra solo unos minutos despues de la migracion.

const bd = db.getSiblingDB("sample_mflix");
const config = db.getSiblingDB("config");
const ns = "sample_mflix.movies";

const hora = new Date().toISOString().slice(11, 19);
print(`=== Distribucion de ${ns} (${hora} UTC) ===`);
const total = bd.movies.countDocuments();
print(`Total de documentos: ${total}\n`);

const col = config.collections.findOne({ _id: ns });

if (!col) {
  // Coleccion sin fragmentar: todo esta en el shard primario de la base
  print("Shard            Documentos      %    Datos (MB)");
  bd.movies.aggregate([{ $collStats: { count: {}, storageStats: {} } }])
    .forEach(s => {
      const pct = total ? (100 * s.count / total).toFixed(1) : "0.0";
```

```
      const mb = (s.storageStats.size / 1048576).toFixed(2);
      print(`${s.shard.padEnd(15)}  ${String(s.count).padStart(10)}   ` +
            `${pct.padStart(5)}  ${mb.padStart(11)}`);
    });
  print("\nLa coleccion todavia no esta fragmentada.");
} else {
  const dist = db.getSiblingDB("admin").aggregate([
    { $shardedDataDistribution: {} },
    { $match: { ns: ns } }
  ]).toArray()[0].shards;

  const chunks = {};
  config.chunks.aggregate([
    { $match: { uuid: col.uuid } },
    { $group: { _id: "$shard", n: { $sum: 1 } } }
  ]).forEach(c => { chunks[c._id] = c.n; });

  dist.sort((a, b) => a.shardName.localeCompare(b.shardName));
  const fila = (c) => c[0].padEnd(15) + c.slice(1)
    .map((v, i) => String(v).padStart([12, 7, 8, 7, 11][i])).join("");
  print(fila(["Shard", "Documentos", "%", "Chunks", "MB", "Huerfanos"]));
  dist.forEach(s => {
    const pct = total ? 100 * s.numOwnedDocuments / total : 0;
    print(fila([s.shardName, s.numOwnedDocuments, pct.toFixed(1),
                chunks[s.shardName] || 0,
                (s.ownedSizeBytes / 1048576).toFixed(1), s.numOrphanedDocs
                   ]));
  });
}

const balanceador = sh.isBalancerRunning();
print(`\nBalanceador activo: ${sh.getBalancerState()}` +
      ` | ronda de balanceo en curso: ${balanceador.inBalancerRound}`);
```

```
docker exec mongos mongosh --quiet --file /scripts/monitoreo/distribucion.
   js
```

```
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB> docker exec mongos mongosh --quiet --file /scripts/monitoreo/distribucio
n.js
=== Distribucion de sample_mflix.movies (00:52:31 UTC) ===
Total de documentos: 21349

Shard           Documentos      %    Datos (MB)
shard2ReplSet        21349  100.0        32.80

La coleccion todavia no esta fragmentada.

Balanceador activo: true | ronda de balanceo en curso: false
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB>
```

Figura 6: Distribución de movies después de la importación.

Como la colección todavía no está fragmentada, las 21 349 películas están en un solo shard, shard2, que es el shard primario de la base sample_mflix.

6 Agregar a los documentos de películas un id_movie

Las películas de sample_mflix solo tienen el _id de tipo ObjectId. Se agregó a cada una un campo id_movie numérico y consecutivo, que después se usa como shard

key. La etapa `$setWindowFields` numera los documentos ordenados por _id con `$documentNumber`, y `$merge` guarda el número en la misma colección.

```
// Paso 5. Agrega a cada pelicula un id_movie entero y consecutivo.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/05-id-movie.js
//
// $setWindowFields numera los documentos ordenados por _id con
// $documentNumber (1, 2, 3, ...). $toInt lo guarda como entero normal y
// $merge escribe el campo en la misma coleccion, buscando cada documento
// por su _id. Se hace antes de fragmentar porque id_movie sera la shard
   key.

const movies = db.getSiblingDB("sample_mflix").movies;

movies.aggregate([
  { $setWindowFields: {
      sortBy: { _id: 1 },
      output: { id_movie: { $documentNumber: {} } } }
  } },
  { $project: { id_movie: { $toInt: "$id_movie" } } },
  { $merge: { into: "movies", on: "_id",
              whenMatched: "merge", whenNotMatched: "discard" } }
]);

const total = movies.countDocuments();
const conId = movies.countDocuments({ id_movie: { $exists: true } });
const rango = movies.aggregate([
  { $group: { _id: null,
              min: { $min: "$id_movie" },
              max: { $max: "$id_movie" },
              distintos: { $addToSet: "$id_movie" } } },
  { $project: { _id: 0, min: 1, max: 1, distintos: { $size: "$distintos" }
     } }
]).toArray()[0];

print(`Peliculas: ${total}`);
print(`Con id_movie: ${conId}`);
print(`id_movie de ${rango.min} a ${rango.max}, ` +
      `${rango.distintos} valores distintos`);

print("\nPrimeras tres peliculas:");
movies.find({}, { _id: 0, id_movie: 1, title: 1, year: 1 })
  .sort({ id_movie: 1 }).limit(3)
  .forEach(m => printjson(m));
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/05-id-
   movie.js
```

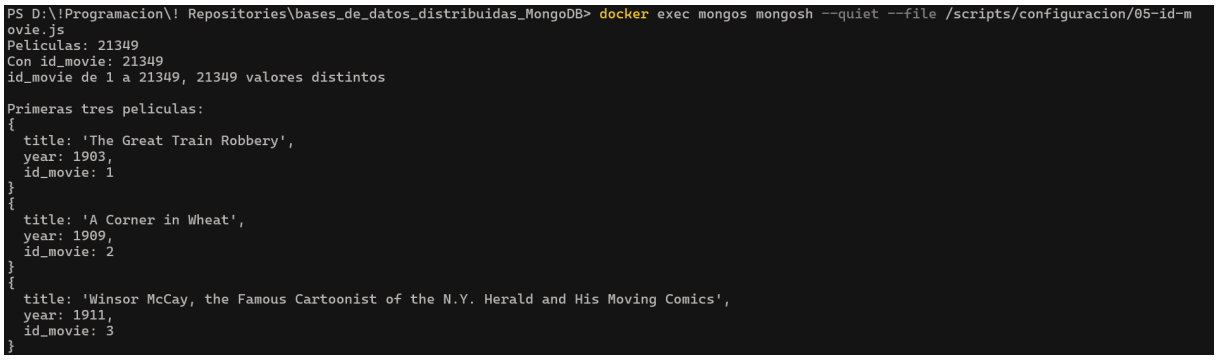


Figura 7: Asignación de id_movie a las películas.

Las 21 349 películas recibieron un id_movie del 1 al 21 349, sin valores repetidos.

7 Definir el Shard Key de la colección Tipo: “Hashed”

La shard key elegida fue `{ id_movie: "hashed"}`. Los criterios fueron los siguientes:

- Cardinalidad y frecuencia: id_movie es único, así que tiene tantos valores como películas y ninguno se repite.
- Crecimiento monótono: id_movie es consecutivo. Con sharding por rangos, las películas nuevas caerían siempre en el último chunk y en un solo shard; el hash reparte los valores consecutivos entre todos los shards.
- Consultas: buscar, actualizar o eliminar una película por su id_movie se envía a un solo shard.

La colección pesa alrededor de 30 MB y el tamaño de chunk por omisión es de 128 MB. Con ese valor la colección completa cabría en un solo chunk y el balanceador no movería nada, por lo que se redujo a 1 MB. Como la colección ya tiene datos, antes de fragmentarla se crea el índice hashed sobre id_movie.

```
// Paso 6. Define la shard key hashed sobre id_movie y fragmenta movies.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/06-shard-collection.js
//
// Shard key: { id_movie: "hashed" }
//  - id_movie es unico: cardinalidad maxima y ningun valor repetido.
//  - Es consecutivo; con sharding por rangos las peliculas nuevas caerian
//    siempre en el ultimo chunk. El hash reparte los valores entre shards.
//  - Buscar, actualizar o borrar por id_movie va a un solo shard.
//
// Tamano de chunk: la coleccion pesa unos 30 MB y el tamano por defecto es
// 128 MB, asi que con ese valor todo se quedaria en un solo chunk y el
// balanceador no moveria nada. Se baja a 1 MB para que los datos se
   repartan.
```

```
const config = db.getSiblingDB("config");
config.settings.updateOne(
  { _id: "chunksize" },
  { $set: { value: 1 } },
  { upsert: true }
);
print(`chunksize: ${config.settings.findOne({ _id: "chunksize" }).value} MB
    `);

const movies = db.getSiblingDB("sample_mflix").movies;

// Para fragmentar una coleccion con datos debe existir el indice
// de la shard key
print(`Indice creado: ${movies.createIndex({ id_movie: "hashed" })}`);

printjson(sh.enableSharding("sample_mflix"));
printjson(sh.shardCollection("sample_mflix.movies", { id_movie: "hashed" })
    );

const info = config.collections.findOne({ _id: "sample_mflix.movies" });
print(`\nShard key de sample_mflix.movies: ${JSON.stringify(info.key)}`);
print("El balanceador empieza a mover datos entre shard1 y shard2;");
print("revisar el avance con monitoreo/distribucion.js");
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/06-shard-
    collection.js
```

```
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB> docker exec mongos mongosh --quiet --file /scripts/configuracion/06-shar
d-collection.js
chunksize: 1 MB
Indice creado: id_movie_hashed
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793840043, i: 4 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793840042, i: 1 })
}
{
  collectionsharded: 'sample_mflix.movies',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793840043, i: 45 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793840043, i: 44 })
}

Shard key de sample_mflix.movies: {"id_movie":"hashed"}
El balanceador empieza a mover datos entre shard1 y shard2;
revisar el avance con monitoreo/distribucion.js
PS D:\!Programacion\! Repositories\bases_de_datos_distribuidas_MongoDB>
```

Figura 8: Definición de la shard key hashed.

Al fragmentar una colección que ya tiene datos, todos los documentos siguen en el shard donde se importaron, y el balanceador empieza a mover rangos hacia el otro shard. Después de unos minutos se ejecutó de nuevo el monitoreo:

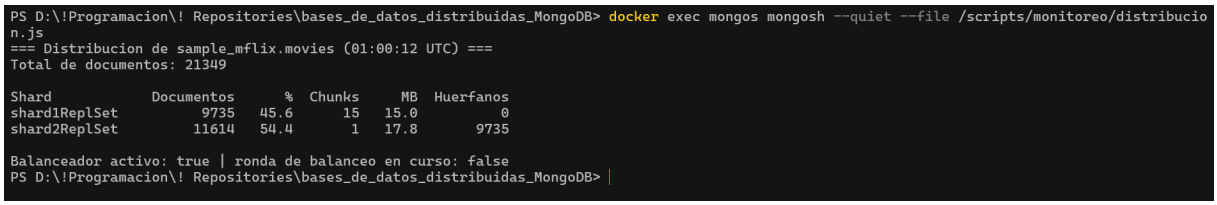


Figura 9: Distribución de movies entre shard1 y shard2 después de fragmentar.

Las películas quedaron repartidas entre shard1 y shard2 en partes similares. El balanceador se detiene cuando la diferencia de datos entre los shards es menor a tres veces el tamaño de chunk, por eso el reparto no es exactamente a la mitad. La columna de huérfanos cuenta las copias que quedan en el shard de origen después de migrar un chunk: `mongos` nunca las devuelve en las consultas, y el shard las borra por sí solo unos minutos después de la migración.

8 Replica sets de los shards

8.1 Estado de los replica sets

Para cada shard se consulta `replSetGetStatus` conectando directo a uno de sus miembros (no a `mongos`, que no expone el estado interno del replica set), con `directConnection` para no depender de que ese miembro sea justo el PRIMARY.

```
// Parte 2 (Diego). Estado de los tres replica sets: un PRIMARY y dos
// SECONDARY por shard, con la hora del ultimo dato replicado en cada uno.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/replicacion/01-estado-replicasets.js
//
// Se conecta directo a un miembro de cada shard (no a mongos, que no
   expone
// el estado del replica set) con directConnection para no depender de que
// ese miembro sea el PRIMARY.

const shards = ["shard1", "shard2", "shard3"];

shards.forEach(nombre => {
  const rs = `${nombre}ReplSet`;
  const uri = `mongodb://${nombre}-1:27017/?directConnection=true`;
  const admin = new Mongo(uri).getDB("admin");
  const estado = admin.runCommand({ replSetGetStatus: 1 });

  print(`${rs}:`);
  estado.members.forEach(m => {
    const optime = m.optimeDate ? m.optimeDate.toISOString() : "-";
    print(`  ${m.name.padEnd(16)} ${m.stateStr.padEnd(10)} optimeDate: ${
       optime}`);
  });
  print("");
});
```

```
docker exec mongos mongosh --quiet --file /scripts/replicacion/01-estado-
    replicasets.js
```

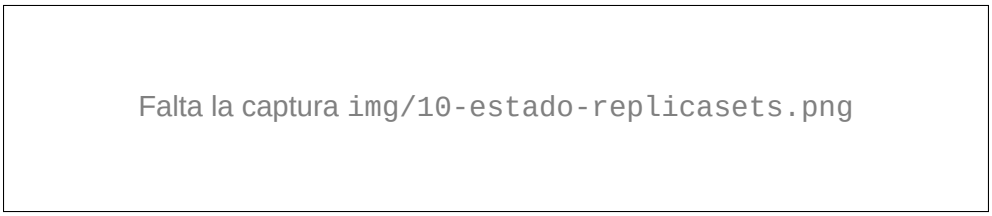


Figura 10: Estado de los tres replica sets: un PRIMARY y dos SECONDARY cada uno.

Los tres shards quedaron con un PRIMARY y dos SECONDARY, con el mismo `optimeDate` entre los miembros de cada uno: no hay retraso de replicación perceptible.

8.2 Los SECONDARY tienen los mismos datos

Se conecta directo al PRIMARY y a un SECONDARY de shard1 y se compara el número de documentos de `sample_mflix.movies` en cada uno. Por defecto un SECONDARY rechaza lecturas para evitar que una aplicación se conecte ahí por error, así que hay que permitirlas explícitamente con `setReadPref("secondary")`.

```
// Parte 2 (Diego). Compara el PRIMARY y un SECONDARY del mismo shard para
// mostrar que el SECONDARY tiene copiados los mismos datos.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/replicacion/02-lectura-secundario.js
//
// Se usa shard1 como ejemplo. shard1-1 es el PRIMARY (priority 2);
// shard1-2 es un SECONDARY. Por defecto un SECONDARY rechaza lecturas para
// evitar que una app se conecte ahi por error, asi que hay que permitirlas
// explicitamente con setReadPref("secondary").

const primario = new Mongo("mongodb://shard1-1:27017/?directConnection=true
    ");
const secundario = new Mongo("mongodb://shard1-2:27017/?directConnection=
    true");
secundario.setReadPref("secondary");

const colPrimario = primario.getDB("sample_mflix").movies;
const colSecundario = secundario.getDB("sample_mflix").movies;

const totalPrimario = colPrimario.countDocuments();
const totalSecundario = colSecundario.countDocuments();

print(`shard1-1 (PRIMARY)   peliculas: ${totalPrimario}`);
print(`shard1-2 (SECONDARY) peliculas: ${totalSecundario}`);

if (totalPrimario === totalSecundario) {
  print("\nEl SECONDARY tiene el mismo numero de documentos que el PRIMARY.
      ");
```

```
} else {
  print("\nLos totales no coinciden (puede que la replicacion siga en curso
     ).");
}

// Un documento cualquiera, comparado tal cual, para confirmar que no solo
// coincide la cantidad sino el contenido.
const muestra = colPrimario.findOne();
const copia = colSecundario.findOne({ _id: muestra._id });
print(`\n¿La pelicula "${muestra.title}" es identica en ambos nodos? ` +
      `${JSON.stringify(muestra) === JSON.stringify(copia)}`);
```

```
docker exec mongos mongosh --quiet --file /scripts/replicacion/02-lectura-
   secundario.js
```

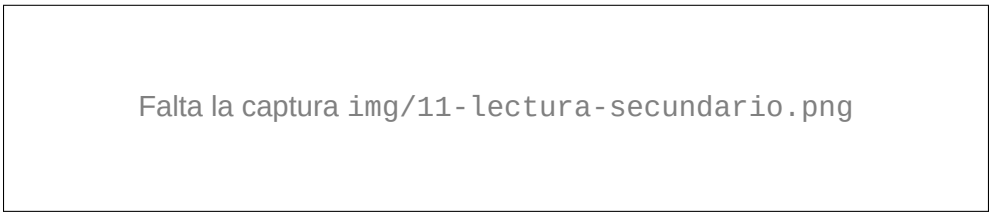


Figura 11: El SECONDARY de shard1 tiene copiados los mismos documentos que su PRIMARY.

El SECONDARY devolvió el mismo total que el PRIMARY, y el documento de ejemplo salió idéntico en ambos: la replicación dentro del shard está al día.

8.3 Prueba de failover

Se detiene el contenedor del PRIMARY de shard1 (`shard1-1`) directamente desde Docker, sin tocar mongosh, y se espera unos segundos a que los dos SECONDARY elijan uno nuevo.

```
docker stop shard1-1
```

```
// Parte 2 (Diego). Failover: correr DESPUES de tumbar al PRIMARY de shard1
   .
//
// En PowerShell, desde la raiz del repo:
//   docker stop shard1-1
//   (esperar unos 10-15 segundos a que los dos SECONDARY elijan uno nuevo)
//   docker exec mongos mongosh --quiet --file /scripts/replicacion/03-
   failover-caida.js
//
// Se consulta el estado desde shard1-2 porque shard1-1 (el que estaba
// conectado antes) ya no responde. Despues se hace una consulta normal a
// traves de mongos para demostrar que el cluster sigue funcionando aunque
// le falte un nodo: mongos solo necesita que cada shard tenga un PRIMARY,
// sin importar cual de los tres sea.
```

```
const admin = new Mongo("mongodb://shard1-2:27017/?directConnection=true").
    getDB("admin");
const estado = admin.runCommand({ replSetGetStatus: 1 });

print("shard1ReplSet (con shard1-1 caido):");
estado.members.forEach(m => print(`  ${m.name.padEnd(16)} ${m.stateStr}`));

const nuevoPrimario = estado.members.find(m => m.stateStr === "PRIMARY");
print(nuevoPrimario
  ? `\nEl nuevo PRIMARY es ${nuevoPrimario.name}.`
  : "\nTodavia no hay PRIMARY, puede que la eleccion siga en curso.");

print("\n--- Consulta a traves de mongos (debe seguir funcionando) ---");
const bd = db.getSiblingDB("sample_mflix");
const total = bd.movies.countDocuments();
print(`Total de peliculas visibles desde mongos: ${total}`);
```

```
docker exec mongos mongosh --quiet --file /scripts/replicacion/03-failover-
    caida.js
```

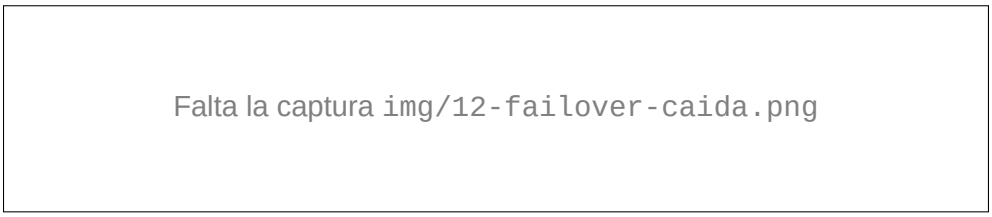


Figura 12: Nuevo PRIMARY elegido en shard1 y consulta a través de mongos funcionando sin shard1-1.

Con `shard1-1` caído, `shard1-2` fue elegido PRIMARY y el cluster, visto desde `mongos`, siguió respondiendo con el total completo de películas: el router no necesita saber cuál nodo específico está activo, solo que el shard tenga un PRIMARY.

Después se vuelve a encender `shard1-1`. Como tiene prioridad más alta, en cuanto termina de ponerse al día provoca una nueva elección y recupera el PRIMARY.

```
docker start shard1-1
```

```
// Parte 2 (Diego). Recuperacion: correr DESPUES de volver a encender
    shard1-1.
//
// En PowerShell:
//   docker start shard1-1
//   docker exec mongos mongosh --quiet --file /scripts/replicacion/04-
    recuperacion.js
//
// shard1-1 vuelve primero como SECONDARY (tiene que ponerse al dia con lo
// que paso mientras estuvo apagado). Como tiene priority 2, en cuanto
// termina de sincronizar provoca una nueva eleccion y recupera el PRIMARY.
// El script espera hasta un minuto mostrando el estado en cada vuelta.

const admin = new Mongo("mongodb://shard1-2:27017/?directConnection=true").
    getDB("admin");
```

```
for (let intento = 1; intento <= 12; intento++) {
  const estado = admin.runCommand({ replSetGetStatus: 1 });
  const shard1_1 = estado.members.find(m => m.name === "shard1-1:27017");

  print(`--- intento ${intento} ---`);
  estado.members.forEach(m => print(`  ${m.name.padEnd(16)} ${m.stateStr}`)
    );

  if (shard1_1 && shard1_1.stateStr === "PRIMARY") {
    print("\nshard1-1 ya recupero el PRIMARY (por su priority 2).");
    break;
  }
  sleep(5000);
}
```

```
docker exec mongos mongosh --quiet --file /scripts/replicacion/04-
   recuperacion.js
```

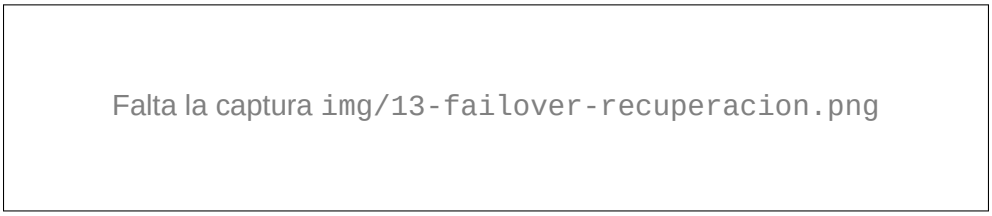


Figura 13: shard1-1 vuelve como SECONDARY y, al ponerse al día, recupera el PRIMARY por su prioridad.

`shard1-1` reapareció primero como SECONDARY (tenía que replicar lo que pasó mientras estuvo apagado) y segundos después, ya sincronizado, volvió a ser PRIMARY — exactamente el comportamiento esperado de una prioridad más alta.

9 Mostrar un ejemplo de balanceo de carga entre shards

Antes de agregar shard3, la colección ya fragmentada entre shard1 y shard2 se revisa con el script de monitoreo.

```
docker exec mongos mongosh --quiet --file /scripts/monitoreo/distribucion.
   js
```

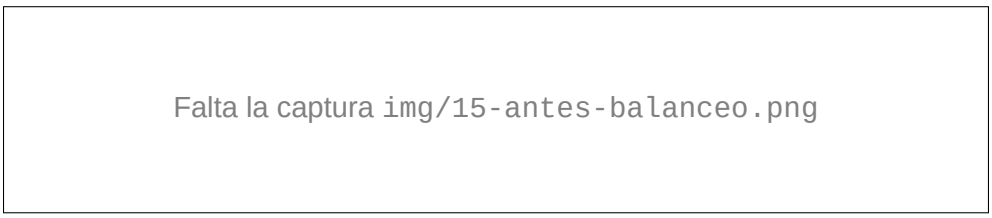


Figura 14: Distribución entre shard1 y shard2 antes de registrar shard3.

`shard3` ya estaba levantado e inicializado como replica set desde el principio (`configuracion/02-init-shards.js`); lo único que faltaba era registrarlo en el cluster con `sh.addShard`.

```
// Parte 2 (Diego). Registra shard3 con la coleccion ya cargada y
   fragmentada
// entre shard1 y shard2, para ver como el balanceador le mueve datos.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/balanceo/01-agregar-shard3.js
//
// shard3 ya estaba levantado e inicializado como replica set desde el
// principio (configuracion/02-init-shards.js); lo unico que faltaba era
// registrarlo en el cluster. En cuanto mongos lo ve disponible, el
// balanceador empieza a moverle chunks por su cuenta, sin que haya que
// pedirselo.

const res = sh.addShard("shard3ReplSet/shard3-1:27017,shard3-2:27017,shard3
   -3:27017");
print(`addShard shard3ReplSet: ok ${res.ok}`);

print("\nShards registrados:");
db.adminCommand({ listShards: 1 }).shards
  .forEach(s => print(`  ${s._id}  ${s.host}`));
```

```
docker exec mongos mongosh --quiet --file /scripts/balanceo/01-agregar-
   shard3.js
```

Falta la captura `img/16-agregar-shard3.png`

Figura 15: shard3 registrado; el cluster ya tiene tres shards.

En cuanto `mongos` ve disponible al shard nuevo, el balanceador empieza a moverle chunks por su cuenta, sin que haya que pedírselo. El avance se sigue corriendo el mismo script de monitoreo varias veces.

```
docker exec mongos mongosh --quiet --file /scripts/monitoreo/distribucion.
   js
```

Falta la captura `img/17-durante-balanceo.png`

Figura 16: Migración en curso: shard3 ya tiene documentos y los otros dos van bajando.

Unos minutos después, con la ronda de balanceo terminada, los tres shards quedan con una parte similar de la colección.



Figura 17: Distribución final entre los tres shards, alrededor de un tercio cada uno.

```
docker exec mongos mongosh --quiet --eval "sh.status()"
```

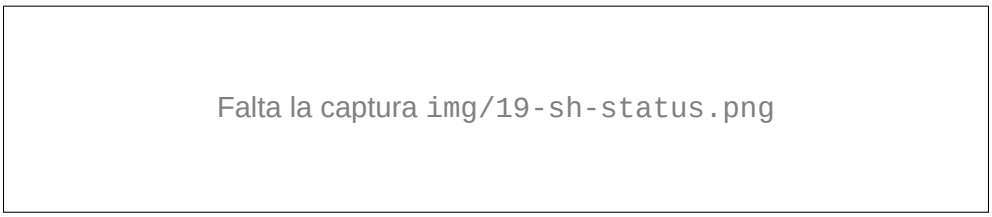


Figura 18: Estado final del cluster con los tres shards registrados.

El reparto no queda exactamente a un tercio cada uno porque el balanceador deja de mover datos en cuanto la diferencia entre shards baja de tres veces el tamaño de chunk (3 MB); aun así, los tres terminan con una proporción parecida de la colección.

10 Realizar operaciones CRUD sobre la base de datos distribuida

10.1 Create

Pendiente: inserción de películas con `insertOne` e `insertMany` (id_movie mayores a 21 349) y monitoreo de la distribución.

10.2 Read

Pendiente: consultas por id_movie (un solo shard) y por otros campos (todos los shards), con `explain()`, y monitoreo.

10.3 Update

Pendiente: actualizaciones con `updateOne` y `updateMany` y monitoreo.

10.4 Delete

Pendiente: eliminaciones con `deleteOne` y `deleteMany` y monitoreo.

11 Conclusión

Pendiente: conclusión del proyecto.

Referencias

Borucki, A. (2025). Using replication and sharding. En *MongoDB 8.0 in Action* (3a ed., cap. 9). Manning.

MongoDB, Inc. (s.f.). *Replication*. MongoDB Manual. https://www.mongodb.com/docs/manual/replication/

MongoDB, Inc. (s.f.). *Sharding*. MongoDB Manual. https://www.mongodb.com/docs/manual/sharding/

udaykirankavaturu. (s.f.). *sharding-demo* [Repositorio]. GitHub. https://github.com/udaykirankavaturu/sharding-demo